

Knowledge representation for efficient learning and communication

Jan Bielecki, Warsaw Python Pizza, 20260509

Learning platforms for computer science

Learning platforms for computer science



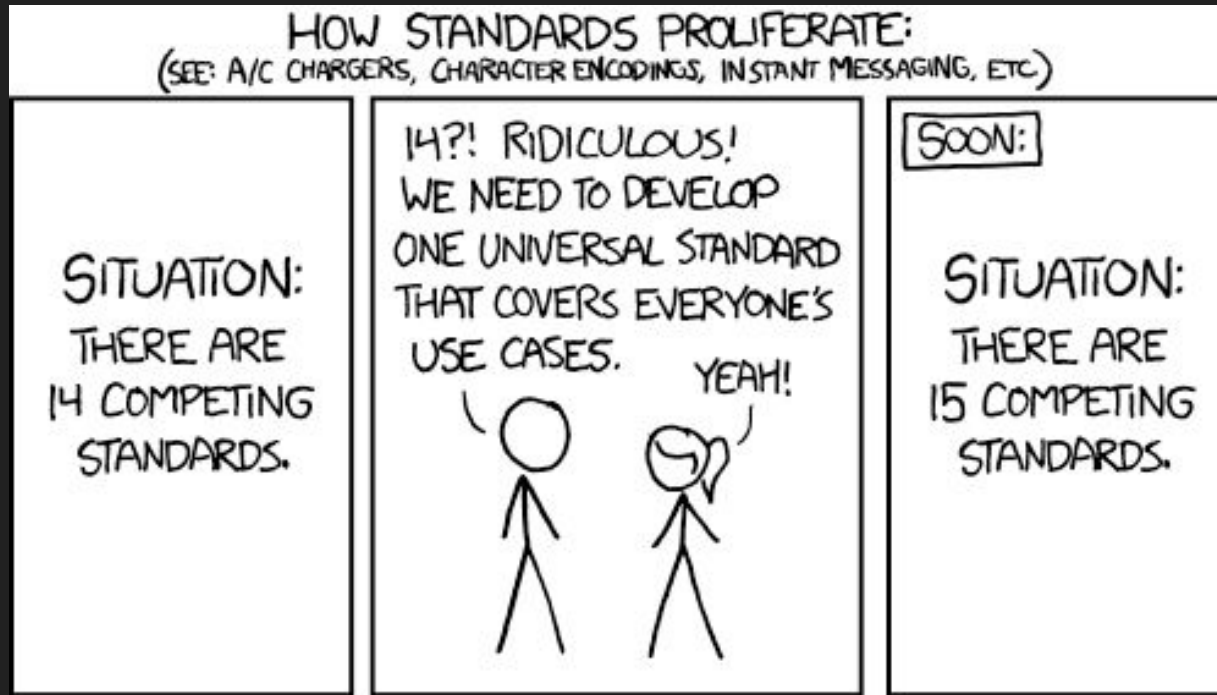
Learning platforms for computer science



Learning platforms for computer science

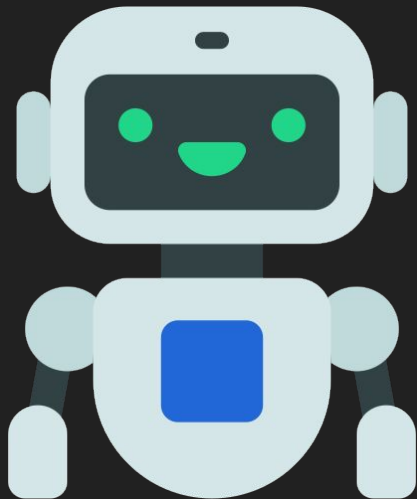


Let's create the universal platform!

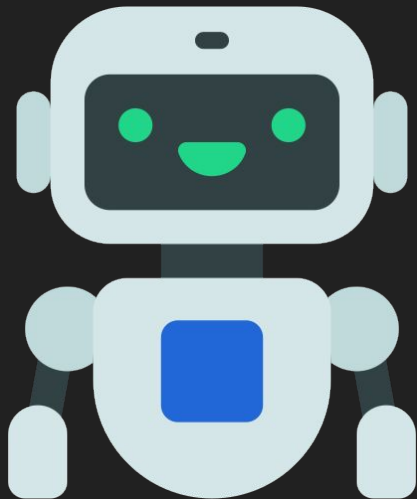


Creating a learning content using LLMs

Creating a learning content using LLMs

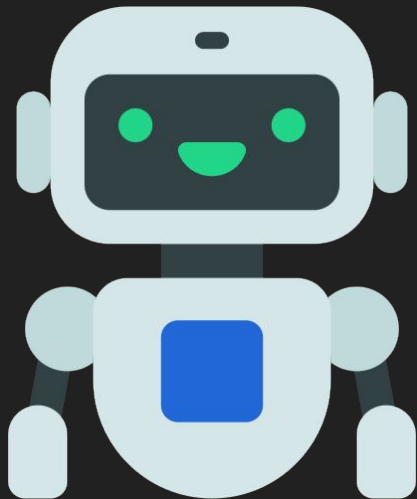


Creating a learning content using LLMs



Looks good on a first sign

Creating a learning content using LLMs

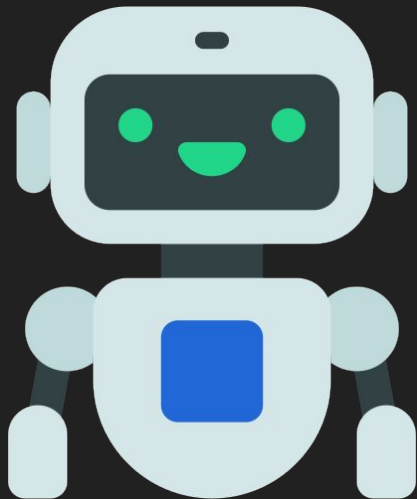


Looks good on a first sign



Fast

Creating a learning content using LLMs



Looks good on a first sign

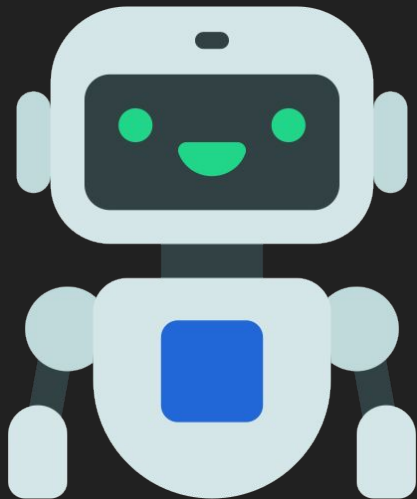


Fast



Questionable quality

Creating a learning content using LLMs



Looks good on a first sign



Fast



Questionable quality



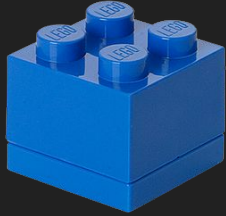
Lack of a architecture/structure

Creating a learning content architecture/structure



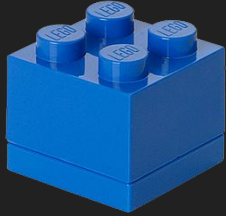
Creating a learning content architecture/structure

Creating a learning content architecture/structure

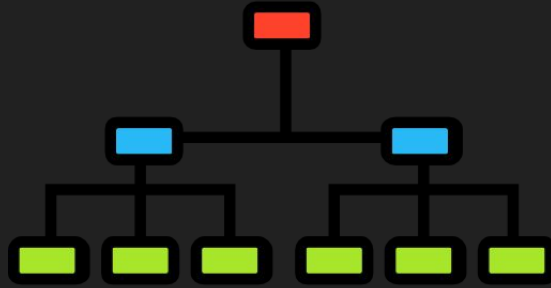


Minimal building block (a
basic unit of knowledge)

Creating a learning content architecture/structure

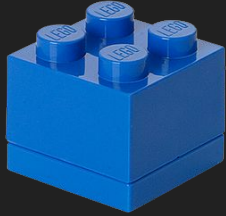


Minimal building block (a
basic unit of knowledge)

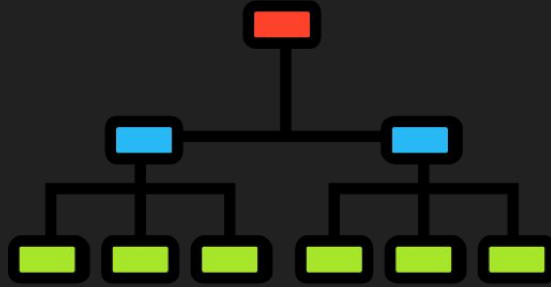


Advancement hierarchy

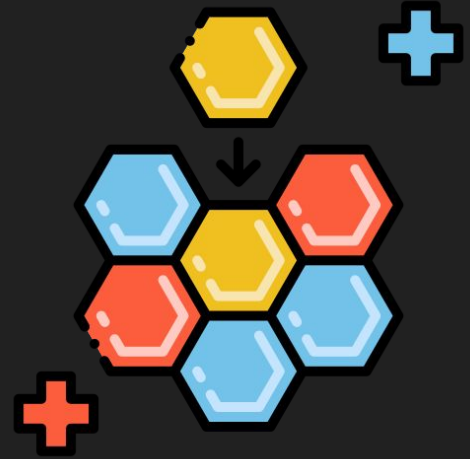
Creating a learning content architecture/structure



Minimal building block (a basic unit of knowledge)

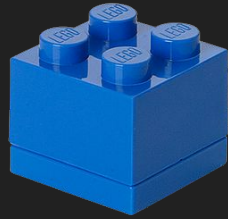


Advancement hierarchy



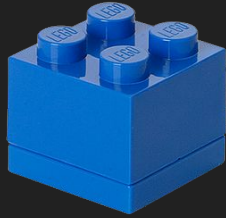
Modularity

Basic unit of knowledge

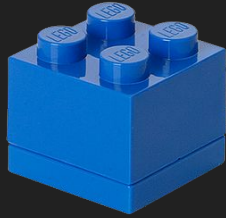


Basic unit of knowledge

Definition (concept/term)



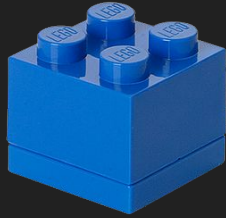
Basic unit of knowledge



Definition (concept/term)

- name

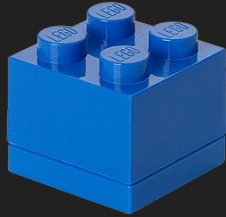
Basic unit of knowledge



Definition (concept/term)

- name
- content (can include other definitions)

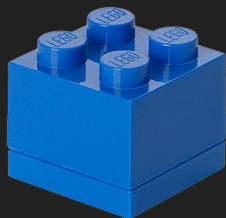
Basic unit of knowledge



Definition (concept/term)

- name
- content (can include other definitions)
- field (a unique key = name + field)

Basic unit of knowledge



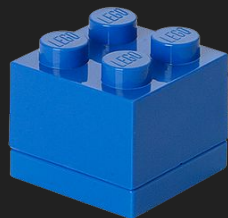
Definition (concept/term)

- name
- content (can include other definitions)
- field (a unique key = name + field)

```
class _FiniteSet(Definition):  
    def _get_content(self) -> str:  
        return f"""
```

```
Finite set is a {SET.key.get_reference(phrase="set")} that has a finite number of elements.  
Informally, a finite set is a set which one could in principle count and finish counting.  
For example, {{2,4,6,8,10}} is a finite set with five elements.  
"""
```

Basic unit of knowledge

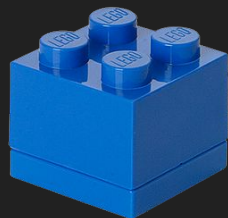


```
class _FiniteSet(Definition):  
    def _get_content(self) -> str:  
        return f"""
```

```
Finite set is a {SET.key.get_reference(phrase="set")} that has a finite number of elements.  
Informally, a finite set is a set which one could in principle count and finish counting.  
For example, {{2,4,6,8,10}} is a finite set with five elements.  
"""
```

```
✓ field  
  > computer_science  
  ✓ mathematics  
    ✓ definitions  
      > algorithm  
    ✓ analysis  
      ✚ __init__.py  
      ✚ asymptotic_behavior.py  
      ✚ bound.py  
      ✚ upper_bound.py  
    > computation  
    > fundamental  
    > graph  
    > notations  
    > problem  
    > tree
```


Basic unit of knowledge



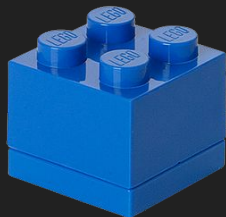
```
class _FiniteSet(Definition):  
    def _get_content(self) -> str:  
        return f"""
```

```
Finite set is a {SET.key.get_reference(phrase="set")} that has a finite number of elements.  
Informally, a finite set is a set which one could in principle count and finish counting.  
For example, {{2,4,6,8,10}} is a finite set with five elements.  
"""
```

```
✓ field  
  > computer_science  
  ✓ mathematics  
    ✓ definitions  
      > algorithm  
    ✓ analysis  
      ✚ __init__.py  
      ✚ asymptotic_behavior.py  
      ✚ bound.py  
      ✚ upper_bound.py  
    > computation  
    > fundamental  
    > graph  
    > notations  
    > problem  
    > tree
```

```
✓ ✓ test  
  ✓ ✓ test_index.py  
    ✓ test_indexes_load_and_length  
    ✓ test_all_definitions_in_indexes  
  ✓ ✓ test_serialize.py  
    ✓ test_generate_and_load  
  ✓ ✓ test_verify_definitions_package  
    ✓ [mathematics]  
    ✓ [computer_science]
```

Basic unit of knowledge

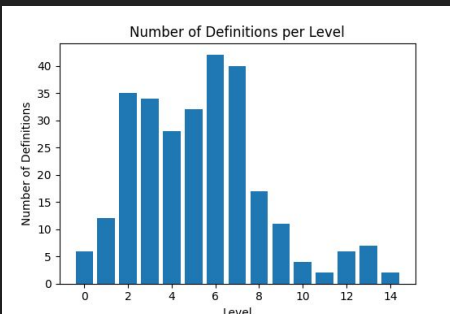


```
class _FiniteSet(Definition):  
    def _get_content(self) -> str:  
        return f"""
```

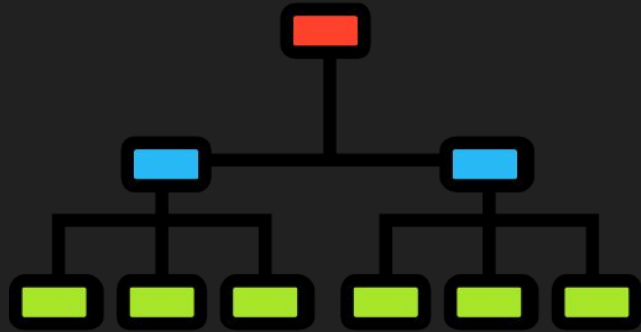
```
Finite set is a {SET.key.get_reference(phrase="set")} that has a finite number of elements.  
Informally, a finite set is a set which one could in principle count and finish counting.  
For example, {{2,4,6,8,10}} is a finite set with five elements.  
"""
```

```
✓ field  
  > computer_science  
  ✓ mathematics  
    ✓ definitions  
      > algorithm  
    ✓ analysis  
      • __init__.py  
      • asymptotic_behavior.py  
      • bound.py  
      • upper_bound.py  
    > computation  
    > fundamental  
    > graph  
    > notations  
    > problem  
    > tree
```

```
✓ ✓ test  
  ✓ ✓ test_index.py  
    ✓ test_indexes_load_and_length  
    ✓ test_all_definitions_in_indexes  
  ✓ ✓ test_serialize.py  
    ✓ test_generate_and_load  
  ✓ ✓ test_verify_definitions_package  
    ✓ [mathematics]  
    ✓ [computer_science]
```

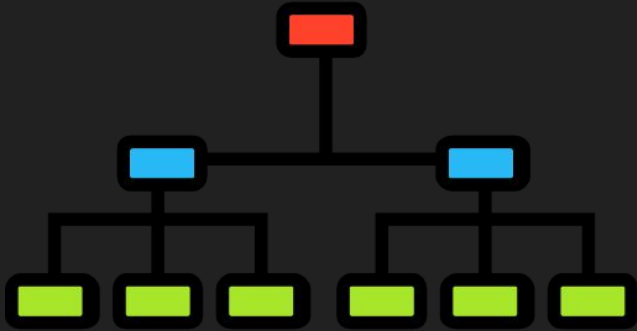


Advancement hierarchy



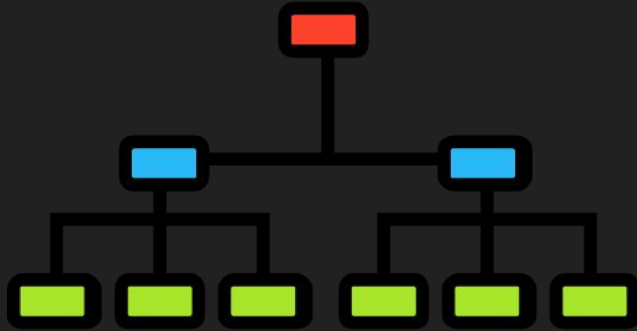
Advancement hierarchy

Definitions together with references (dependencies) form a graph.



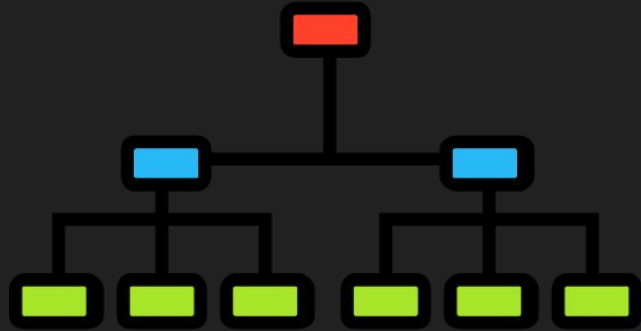
Advancement hierarchy

Definitions together with references (dependencies) form a graph.



1) Definition -> node

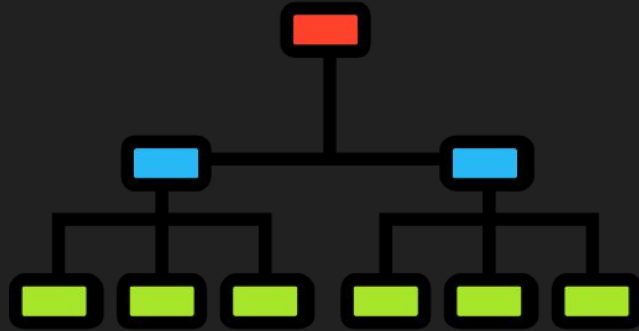
Advancement hierarchy



Definitions together with references (dependencies) form a graph.

- 1) Definition -> node
- 2) Dependency -> directed edge

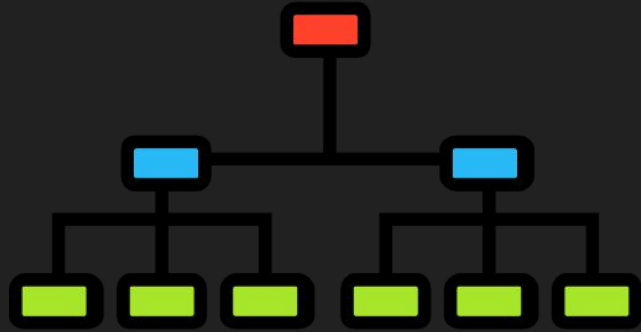
Advancement hierarchy



Definitions together with references (dependencies) form a graph.

- 1) Definition -> node
- 2) Dependency -> directed edge
- 3) Rule -> no cycles

Advancement hierarchy

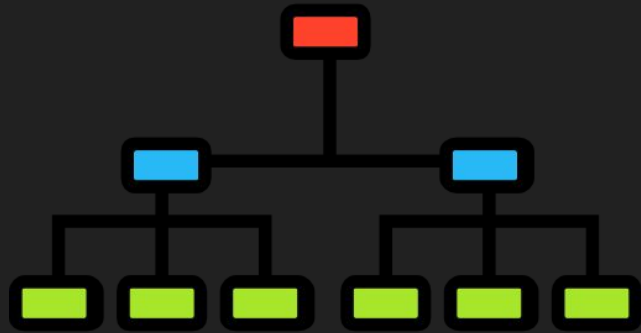


Definitions together with references (dependencies) form a graph.

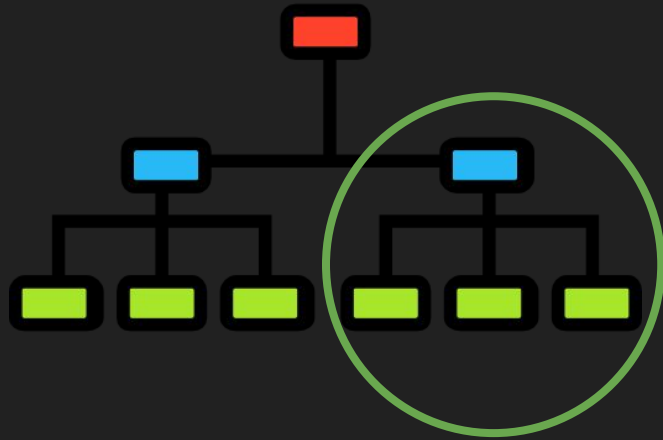
- 1) Definition -> node
- 2) Dependency -> directed edge
- 3) Rule -> no cycles

Directed Acyclic Graph (DAG)

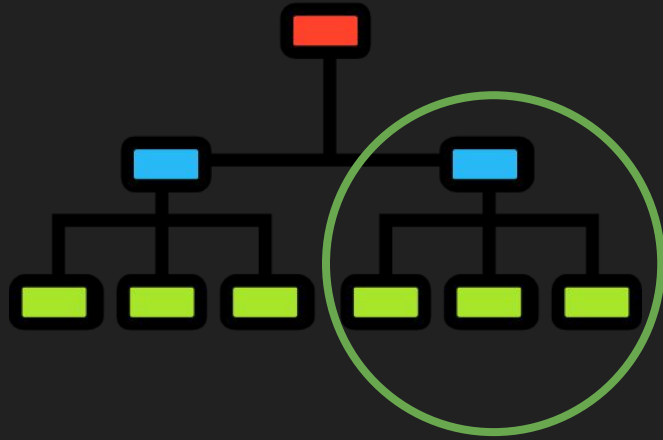
Modularity



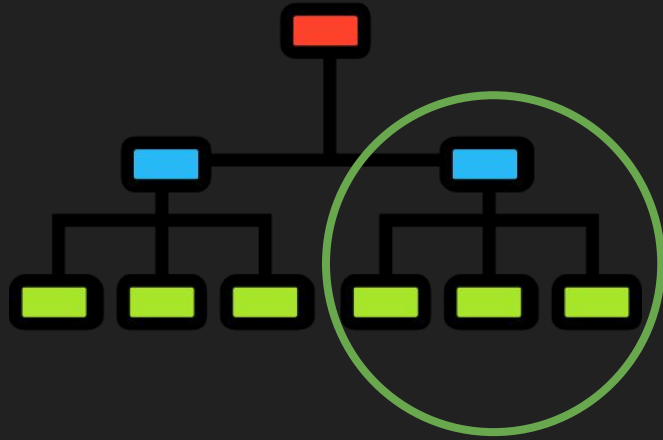
Modularity



Modularity

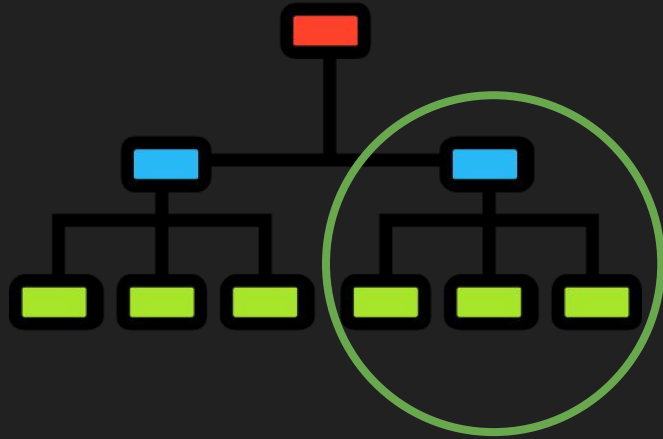


Modularity



Data Structures & Algorithms

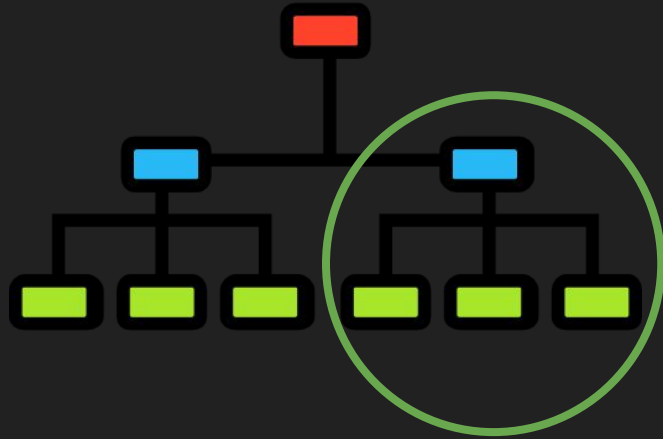
Modularity



Data Structures & Algorithms

Computer hardware

Modularity



Data Structures & Algorithms

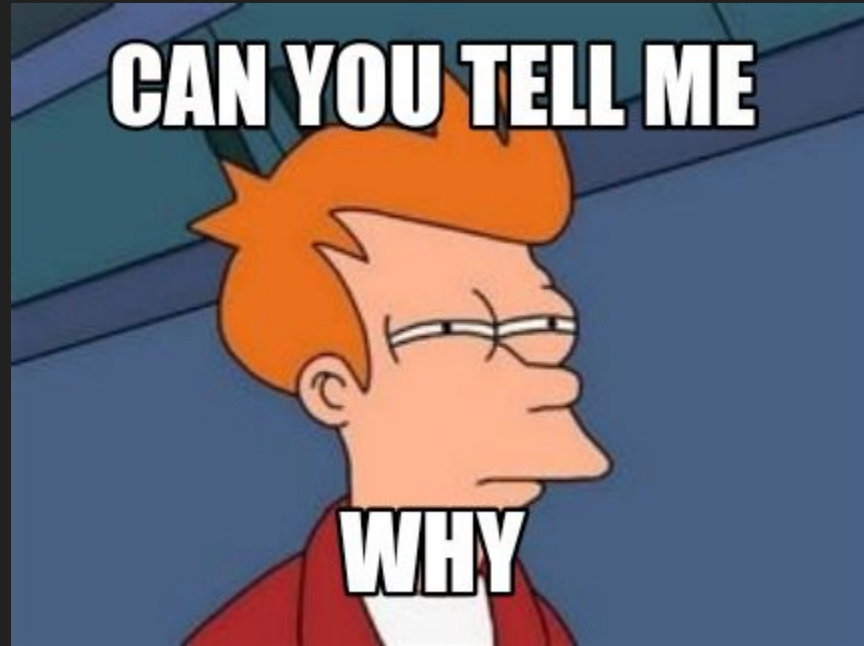
Computer hardware

Databases

[illegible]

k4liber.github.io/definit-dsa

What is the value?



What distinguishes this approach from existing ones?

What distinguishes this approach from existing ones?

- 1) Breaking down (decompose) complex topics into small “knowledge pills”.

What distinguishes this approach from existing ones?

- 1) Breaking down (decompose) complex topics into small “knowledge pills”.
- 2) DAG structure suggests “where to start”.

What distinguishes this approach from existing ones?

- 1) Breaking down (decompose) complex topics into small “knowledge pills”.
- 2) DAG structure suggests “where to start”.
- 3) A living and high-quality glossary for any field of knowledge used as reference or ML training data.

The biggest challenges



The biggest challenges

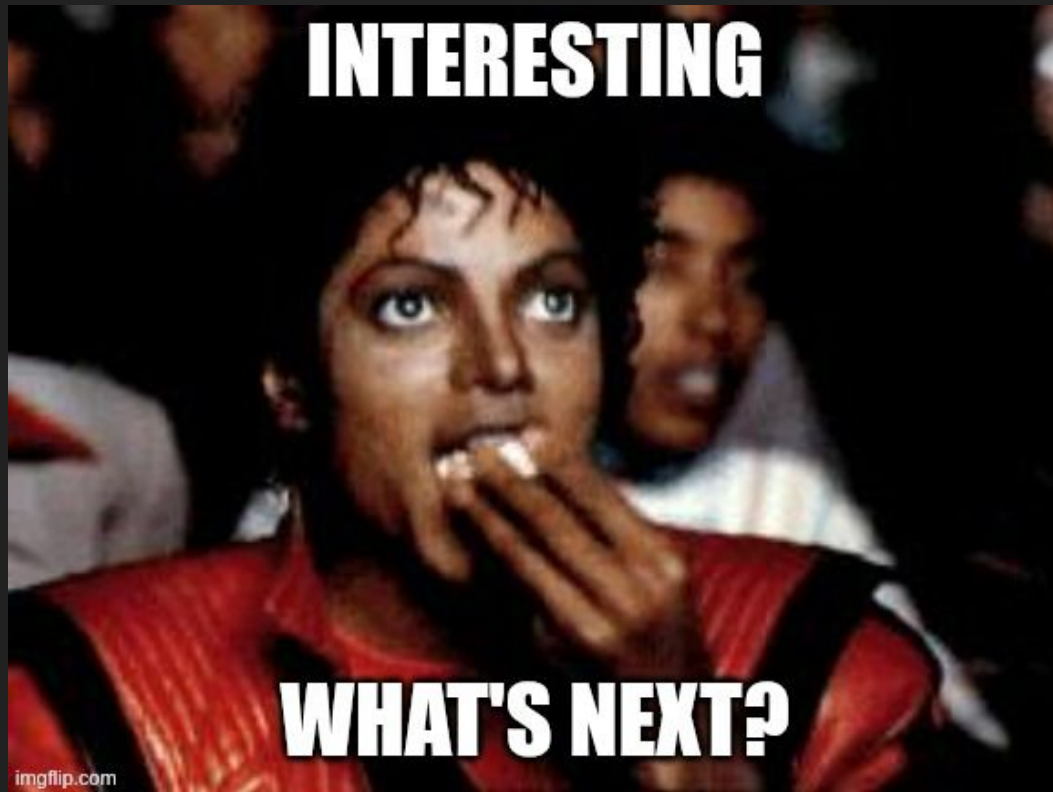
The biggest challenges

- 1) DAG structure is an assumption that all knowledge can be represented in a hierarchical manner.

The biggest challenges

- 1) DAG structure is an assumption that all knowledge can be represented in a hierarchical manner.
- 2) "In theory, there is no difference between theory and practice. But in practice, there is."~Benjamin Brewster

What's next?



Facing the challenges

- 1) DAG structure is an assumption that all knowledge can be represented in a hierarchical manner.
- 2) "In theory, there is no difference between theory and practice. But in practice, there is."~Benjamin Brewster

Facing the challenges

- 1) DAG structure is an assumption that all knowledge can be represented in a hierarchical manner.

Creating next modules to verify the assumption.

- 2) "In theory, there is no difference between theory and practice. But in practice, there is."~Benjamin Brewster

Facing the challenges

- 1) DAG structure is an assumption that all knowledge can be represented in a hierarchical manner.

Creating next modules to verify the assumption.

- 2) "In theory, there is no difference between theory and practice. But in practice, there is."~Benjamin Brewster

Vibe-coding gamification aspects: quests, badges, ranks.

"You can outsource your thinking, but you cannot outsource your understanding."



github.com/K4liber/definit